

Analizador de Código Fonte

Engenharia Gramatical | Ano Letivo 2024/2025

Maria Cunha
Mestrado em Engenharia Informática
Universidade do Minho
Braga, Portugal
pg54042@alunos.uminho.pt

Tomás Campinho
Mestrado em Engenharia Informática
Universidade do Minho
Braga, Portugal
pg57742@alunos.uminho.pt

Resumo—Este documento visa explicar um analisador desenvolvido para interpretar uma linguagem LPI concebida no âmbito da unidade curricular de Engenharia Gramatical. Assim, a primeira secção focar-se-á brevemente a explicar os elementos essenciais da LPI desenvolvida, fornecendo contexto para os passos tomados na conceção do analisador. De seguida, explicar-se-á em detalhe como o analisador foi realizado, tendo em conta as funcionalidades desejadas deste, definidas pelos docentes. Finalmente, fazer-se-á uma curta análise crítica ao trabalho desenvolvido.

I. LPI DESENVOLVIDA

A Linguagem de Programação Imperativa, ou LPI, foi desenvolvida com o objetivo de proporcionar uma sintaxe simples e acessível. Esta permite a escrita de programas estruturados, com declaração explícita de variáveis e coleções, a definição de funções e controlo de fluxo através de instruções comuns em linguagens modernas.

A LPI está dividida em duas grandes secções: **declarações** e **instruções**. Todas as declarações - sejam de variáveis, coleções ou funções - devem ser feitas no início do programa (por outras palavras, todas as declarações devem ser feitas antes de qualquer atribuição, estrutura de controlo, etc). Esta separação clara entre declarações e instruções garante uma estrutura bem definida e facilita a análise estática do código.

A linguagem suporta:

- **Tipos de dados simples:** como inteiros, *strings* e booleanos;
- **Coleções estruturadas:** como *arrays*, *sets* e *tuples*;
- **Definição de funções:** com parâmetros, tipo de retorno, e corpo;
- **Instruções comuns:** como atribuições, leitura e escrita, condicionais (*if*, *case*) e ciclos (*while*, *repeat*, *for*);
- **Expressões compostas:** que incluem operadores aritméticos, chamadas de função e manipulação de coleções com operadores como *cons*, *snoc*, *in*, *head* e *tail*.

II. DESENVOLVIMENTO DO ANALISADOR

Este trabalho prático consistiu no desenvolvimento de um analisador estático para a Linguagem de Programação Impe-

rativa previamente definida, conforme os requisitos especificados. A solução é implementada em *Python* e utiliza o módulo *Lark* para processamento de linguagens.

O sistema foi estruturado em três componentes principais:

- 1) **Processamento Sintático:** Realizado pelo *parser* gerado a partir da gramática LPI definida no ficheiro *lpi.lark*. Esta etapa transforma o código fonte numa árvore sintática abstrata.
- 2) **Análise Semântica:** Implementada através da classe *AnalizadorLPI*, que percorre a árvore sintática abstrata gerada, utilizando o módulo *Visitor*. Durante esta travessia, são armazenadas todas as informações necessárias para atender aos requisitos do enunciado.
- 3) **Geração do Relatório:** A fase final transforma os dados recolhidos e apresenta-as através de um ficheiro *HTML* estruturado, organizado conforme os cinco pontos principais solicitados.

A. Implementação dos Requisitos

1) **Gestão de Variáveis:** O sistema integra um mecanismo de monitorização de variáveis, permitindo acompanhar de forma precisa todas as fases da sua utilização no programa.

- **Hierarquia de Contextos:** O sistema distingue de forma rigorosa entre o contexto global e os contextos locais associados às funções, garantindo que as variáveis são corretamente associadas ao respetivo âmbito de definição e evitando ambiguidades na sua interpretação;
- **Estado das Variáveis:** Para cada variável, é realizada a monitorização das suas principais fases de vida, nomeadamente a declaração, a inicialização e a utilização. Este acompanhamento permite detetar eventuais situações de uso de variáveis não inicializadas ou declaradas mas nunca utilizadas, contribuindo para uma análise mais fiável do código;
- **Conflitos de Nomes:** O sistema inclui mecanismos de deteção de conflitos, sinalizando a redeclaração de variáveis dentro do mesmo contexto, o que permite prevenir erros de lógica e reforçar a integridade do espaço de nomes do programa analisado.

A implementação recorre a várias estruturas de dados complementares para garantir um controlo rigoroso sobre o estado das variáveis ao longo do programa:

```
self.vars_declaradas = {}
self.vars_redeclaradas = set()
self.vars_ao_declaradas = {}
self.vars_ao_inicializadas = set()
self.vars_atribuidas = set()

self.scope_atual = "global"
self.scopes = {"global": {}}
self.funcoes_declaradas = {}

self.vars_atribuidas_por_scope = {"global":
    set()}
self.vars_usadas_por_scope = {"global": set()}
```

Cada uma destas estruturas desempenha um papel específico: a variável `vars_declaradas` mantém o registo das variáveis declaradas no contexto global; A variável `vars_atribuidas` acompanha as variáveis que foram efetivamente inicializadas com um valor; As variáveis que dizem respeito aos *scopes* organizam os diferentes contextos de execução, permitindo o controlo de variáveis locais a funções;

Esta abordagem permite identificar precisamente os casos de:

- Redefinição (variável declarada duas vezes no mesmo contexto);
- Uso sem declaração (variável referenciada sem declaração prévia);
- Declaração sem uso (variável declarada mas nunca referenciada);
- Uso sem inicialização (variável referenciada sem ter sido atribuído um valor).

2) *Classificação por Tipos de Dados*: O analisador mantém um dicionário completo das variáveis classificadas por tipo.

Para cada declaração encontrada (seja de tipo básico ou coleção), a variável é registada na lista correspondente ao seu tipo. Esta informação é depois processada para gerar a tabela de frequências por tipo de dado, incluindo:

- Tipos primitivos (*int*, *string*, *bool*)
- Estruturas de dados (*array*, *set*, *tuple*)
- Tipos compostos (*array int*, *set string*, etc.)

3) *Estatísticas de Fluxo de Controlo*: O sistema conta todas as instruções do programa, classificadas em categorias:

- **Atribuições**: Operações de atribuição de valores.
- **Read/Write**: Operações de leitura e escrita.
- **Condicionais**: Estruturas *if* e *case*.
- **Cíclicas**: *Loops* *while*, *repeat* e *for*.

A contagem é realizada através de incrementos nos métodos *visitors* correspondentes. Como, por exemplo, é o caso no seguinte trecho de código:

```
def assign(self, arvore):
    self.instrucoes["atribuicoes"] += 1
    ...
```

4) *Análise de Estruturas Aninhadas*: Para avaliar a complexidade do fluxo de controlo, o analisador implementa um sistema de *stack* que monitora:

- A profundidade de aninhamento.
- Os tipos de estruturas envolvidas.
- As relações entre estruturas de controlo.

Através dos contadores `nivel_controlo` e `nivel_controlo_geral`, o sistema pode identificar quando uma estrutura de controlo aparece dentro de outra, registando estas ocorrências para o ficheiro *HTML*.

5) *Otimização de Condicionais*: O sistema desenvolvido para otimizar instruções condicionais utiliza estruturas complementares:

```
self.ifs_simplificaveis = []
self.parent_if_stack = []
self.parent_if_conditions = []
self.current_if_level = 0
```

Estas estruturas são usadas para detetar padrões de condicionais aninhados que podem ser simplificados.

O processo de otimização segue os seguintes passos:

- Cada vez que um *if* é encontrado, a condição é extraída através do método `_get_condition_text`;
- Se o novo *if* estiver aninhado diretamente no bloco *then* de outro *if* (i.e., não há instruções entre os dois e não existe um *else* associado ao *if* pai), a combinação das duas condições é considerada;
- O par de condições (*parent* e *child*) é registado na lista `ifs_simplificaveis`, juntamente com duas representações:
 - A forma original (com dois *ifs* aninhados);
 - A forma otimizada (um único *if* com as duas condições combinadas por um operador *and*).
- Para garantir que a otimização é segura, o sistema verifica o nível de controlo geral e o contexto atual através de variáveis como `nivel_controlo_geral` e `current_if_level`.

O objetivo desta análise é identificar padrões como:

```
if condicao1:
    if condicao2:
        # ...
```

e sugerir a sua reescrita para:

```
if condicao1 and condicao2:
    # ...
```

B. Ficheiro HTML

O método `gerar_analise` produz um documento *HTML* completo com:

- 1) **Gestão de Variáveis:** Uma lista organizada de todos os problemas identificados com variáveis;
- 2) **Distribuição por Tipo:** Uma tabela com o número total de variáveis declaradas por tipo;
- 3) **Estatísticas de Código:** A contagem de instruções por categoria;
- 4) **Complexidade Estrutural:** A quantidade de vezes em que ocorre aninhamento de estruturas de controlo;
- 5) **Oportunidades de Otimização:** Sugestões para simplificação de condicionais.

C. Exemplo de Utilização

Um dos exemplos de código da nossa LPI utilizados para teste:

```
int x
int y
array int numeros
int primeiro
string m
array int numeros
function dobro(int n): int
    return n * 2
end
function aumenta(int a): string
    string mensagem
    while dobro(a) > a do
        if ( a / 2 ) == 0 then
            a = a + 2
        else
            a = a + 3
        end
    end
    mensagem = "Aumentei_a"
    return mensagem
end

numeros[0] = x
if not numeros == [ ] then
    if in(3, numeros) then
        write("3_está_na_coleção")
    end
end
primeiro = head(numeros)
y = dobro(segundo)
mensagem = aumenta(y)
```

O ficheiro *HTML* resultante, possível de visualizar através da Figura 1 da secção *Anexos*, apresenta as seguintes informações sobre o exemplo de código:

1. Gestão de Variáveis

- **Variáveis Redeclaradas:** *numeros*
- **Variáveis Não Declaradas:** *mensagem [global], segundo*

- **Variáveis Usadas Mas Não Inicializadas:** *x*
- **Variáveis Declaradas Mas Não Mencionadas:** *m*

2. Classificação por Tipos de Dados

Tipo	Variáveis Declaradas	Total
int	<i>x, y, primeiro</i>	3
array int	<i>numeros</i>	1
string	<i>m, mensagem [função: aumenta]</i>	2

3. Estatísticas de Código

- Atribuições: 7
- Leituras: 0
- Escritas: 1
- Condicionais: 3
- Cíclicas: 1

4. Estruturas Aninhadas

- Total de situações em que estruturas surgem aninhadas: 2

5. Oportunidades de Otimização

Ifs aninhados que podem ser substituídos por um só if:

- if not numeros == [] then
 if in(3, numeros) then

Pode ser substituído por

if not numeros == [] and in(3, numeros) then

III. CONCLUSÃO

Ao longo do desenvolvimento deste trabalho, o grupo reconheceu que existiram oportunidades de otimização, tanto ao nível do código do analisador como nas regras inicialmente definidas para a gramática da linguagem LPI. À medida que se trabalhava com exemplos de código mais diversificados para testar o analisador, tornaram-se evidentes algumas limitações na gramática, como, por exemplo, a ausência de um operador *not* para condições, o que nos levou a fazer umas ligeiras mudanças em relação à linguagem desenhada inicialmente.

Em termos de trabalho futuro, reparamos que no tratamento de *ifs* aninhados será possível fazer com que o analisador de código os compare para a hipótese de poder ser feita a remoção de condições que já são cobertas por outras.

Por fim, apesar destes desafios e possíveis melhorias a fazer, o grupo considera que cumpriu todos os objetivos propostos no enunciado do trabalho prático e está satisfeito com o resultado final do projeto.

Análise de Programa LPI

```

int x
int y
array int numeros
int primeiro
string m
array int numeros
function dobro(int n): int
    return n * 2
end
function aumenta(int a): string
    string mensagem
    while dobro(a) > a do
        if ( a / 2 ) == 0 then
            a = a + 2
        else
            a = a + 3
        end
    end
end
mensagem = "Aumentei a"
return mensagem
end

numeros[0] = x
if not numeros == [ ] then
    if in(3, numeros) then
        write("3 está na coleção")
    end
end
primeiro = head(numeros)
y = dobro(segundo)
mensagem = aumenta(y)

```

1. Gestão de Variáveis

Variáveis Redefinidas: *numeros*

Variáveis Não Declaradas: *mensagem* [global], *segundo*

Variáveis Usadas Mas Não Inicializadas: *x*

Variáveis Declaradas Mas Não Mencionadas: *m*

2. Classificação por Tipos de Dados

Tipo	Variáveis Declaradas	Total
int	<i>x, y, primeiro</i>	3
array int	<i>numeros</i>	1
string	<i>m, mensagem</i> [função: <i>aumenta</i>]	2

3. Estatísticas do Código

- Atribuições: 7
- Leituras: 0
- Escritas: 1
- Condicionais: 3
- Cíclicas: 1

4. Estruturas Aninhadas

Total de situações em que estruturas surgem aninhadas: 2

5. Oportunidades de Otimização

Ífs aninhados que podem ser substituídos por um só if:

- if not numeros == [] then
 if in(3, numeros) then

Pode ser substituído por

```
if not numeros == [ ] and in(3, numeros) then
```

Figura 1: Página de Análise HTML